

Operating systems

Threads Programatically

Content Lecture#7

01 Threads

02 Benefits of Threads

03 Why Threads

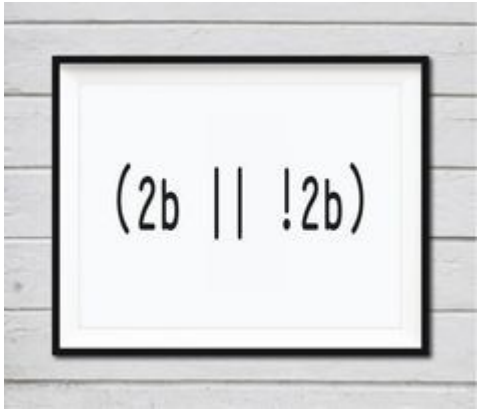
04 Threads VS Process

05 S

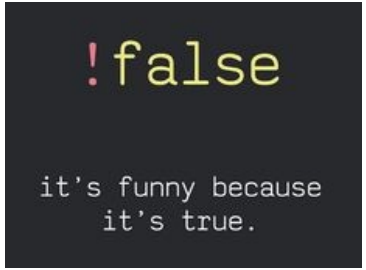
Lets take a Break

A programmer had a problem, so he used threads. Then he had two problems.









When the code is a mess but it's working anyway





Threads

Flow of Execution
Basic Unit of execution

POSIX

Postable Operating System interface for Unix (POSIX),

Creating POSIX Threads

- Name top

  Pthread_create– CREATE A NEW THREAD

- Synopsis

  #include<pthread.h>

  Int pthread_create(pthread_t *thread , const pthread_attr_t *attr, void *(*start_routine) (void *), void *arg);

  Int pthread_create (ID, Attributed, functions extending thread, arguments)

  Compile and link with –pthread.

- 1st parameter is pointer to pthread_t in which thread is returned Thread ID
- 2nd parameter attributes if not set then pass NULL means default, like stack size, scheduling
- 3rd parameter is function from where it is extended. function pointer give return type void pointer (* start routine function passed by reference , void * passed as argument which is only one)
- 4th argument passed otherwise NULL means no argument

Joining Threads

- Name
  - Pthread_join – join with a terminated thread
- SYNOPSIS
  - #include <pthread.h>
  - Int pthread_join(pthread_t thread, void **retval);

  Compile and link with -pthread

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
#include <wait.h>
#include <pthread.h>
void* f1(void* ptr)
{
        while(1)
                printf("Thread 1");

}
void* f2(void* ptr)
{
        while(1)
                printf("Thread 2");

}
int main()
{
        pthread_t tid1,tid2;
        pthread_create(&tid1,NULL,&f1,NULL);
        pthread_create(&tid2,NULL,&f2,NULL);
        pthread_join(tid1,NULL);
        pthread_join(tid2,NULL);
return 0;
}
```

Threads Types

Kernel threads

- Theses are supported directly by operating system
- The kernel perform thread creation, scheduling and management
- Example kernel threads supporting OS are;
  - Windows 95/98/2000/XP
  - Solaris, Tru64, Unix
  - Linux, Mach and OS/2

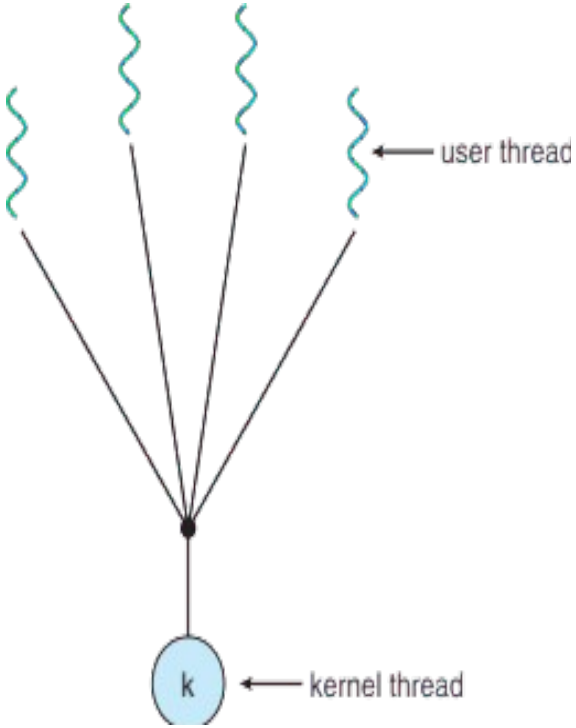
User threads

- These are supported above the kernel, via a set of library calls at the user level
- Library provides support for thread creation, scheduling and management with no support from kernel
- Example user-threads
  - POSIX P thread
  - Mach C thread
  - Solaris UI thread
- Hybrid approach implements both user-level and kernel supported threads

Multithreading Models

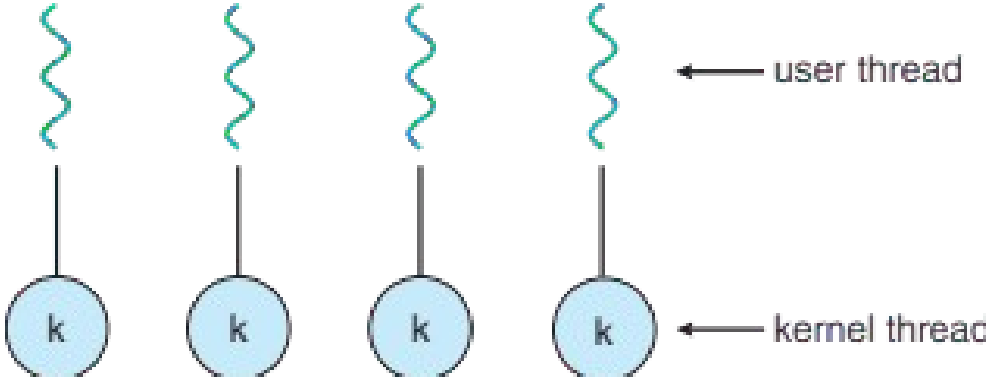
Many to One Model

- User level thread are easy to manage or create
- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Used on systems that do not support kernel threads
- User threads are not given separate time slice. They have to share the time slice given to the kernel level thread they are mapped on
- Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
  - Solaris Green Threads
  - GNU Portable Threads



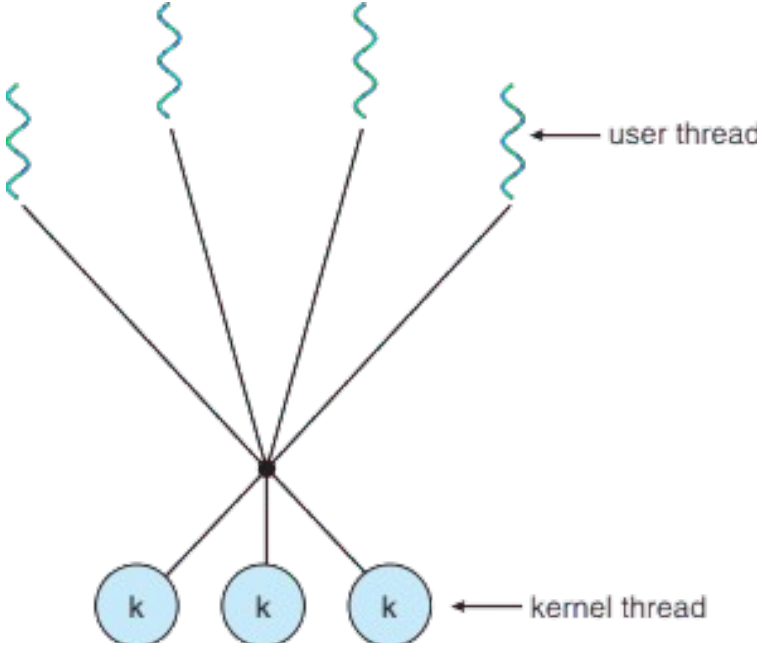
Multithreading Models

- One to One Model
- Each user-level thread maps to kernel thread
- Each thread is assigned a separate time slice
- Creating a user-level thread creates a kernel thread
- Block one thread transfer control to other
- Number of threads per process sometimes restricted due to overhead
- Not portable
- Create/Cancel and synchronization need kernel interventions
- Examples
  - Windows
  - Linux
  - Solaris 9 and later



Multithreading Models

- Many to Many Model
- Allow many user level threads to map to many kernel threads
- Allow the operating system to create a sufficient number of kernel threads
- Example OS;
  - Windows NT/2000 with thread fiber package
  - Windows XP, 7
  - Solaris prior to version 9



Threading Issues

The fork() and exec() System Calls

- The fork() system call is used to create a separate, duplicate process.
- The exec() family of functions **replaces** the current process image with a new process image.

- The semantics of the fork() and exec() system calls change in a multithreaded program.

- In a program calls fork(). Some UNIX systems have chosen to have two versions of fork()
  - One that duplicates all threads.
  - Another that duplicates only the thread that invoked the fork() system call.
- A thread invokes the exec() system call, the program specified in the parameter to exec() will replace the entire process—including all threads.
- if exec() is called immediately after forking, then duplicating all threads is unnecessary, as the program specified in the parameters to exec() will replace the process.

Threading Issues

A **signal i**s used in UNIX systems to notify a process that a particular event has occurred.

1. A signal is generated by the occurrence of a particular event.
2. The signal is delivered to a process.
3. Once delivered, the signal must be handled.

A signal may be handled by one of two possible handlers:

1. A default signal handler
2. A user-defined signal handler

Every signal has a default signal handler that the kernel runs when handling that signal. This default action can be overridden by a user-defined signal handler that is called to handle the signal.

Handling signals in single-threaded programs is straightforward: signals are always delivered to a process. However, delivering signals is more complicated in multithreaded programs

1. Deliver the signal to the thread to which the signal applies.
2. Deliver the signal to every thread in the process.
3. Deliver the signal to certain threads in the process.
4. Assign a specific thread to receive all signals for the process

Synchronous signals need to be delivered to the thread causing the signal and not to other threads in the process.

Some asynchronous signals—such as a signal that terminates a process (<control><C>, for example)—should be sent to all threads

Threading Issues

Thread cancellation

Thread cancellation involves terminating a thread before it has completed.

A thread that is to be canceled is often referred to as the target thread. Cancellation of a target thread may occur in two different scenarios:

1. Asynchronous cancellation. One thread immediately terminates the target thread.
2. Deferred cancellation. The target thread periodically checks whether it should terminate, allowing it an opportunity to terminate itself in an orderly fashion.

The difficulty with cancellation occurs in situations where resources have been allocated to a canceled thread or where a thread is canceled while in the midst of updating data it is sharing with other threads especially **synchronous cancellation.**

The default cancellation type is deferred cancellation. Here, cancellation occurs only when a thread reaches a cancellation point. One technique for establishing a cancellation point is to invoke the pthread testcancel() function. If a cancellation request is found to be pending, a function known as a cleanup handler is invoked.

Implicit Threading

Growth of multicore processing, applications containing hundreds—or even thousands—of threads

address these difficulties and better support the design of multithreaded applications is to transfer the creation and management of threading from application developers to compilers and run-time libraries.

This strategy, termed implicit threading

- Thread Pools
- OpenMP
- Grand Central Dispatch

Thread Pools

The size of a thread pool is the number of threads kept in reserve for executing tasks. It is usually a tunable parameter of the application, adjusted to optimize program performance. Deciding the optimal thread pool size is crucial to optimize performance.

he general idea behind a thread pool is to create a number of threads at process startup and place them into a pool, where they sit and wait for work. When a server receives a request, it awakens a thread from this pool—if one is available—and passes it the request for service. Once the thread completes its service, it returns to the pool and awaits more work. If the pool contains no available thread, the server waits until one becomes free.

Thread pools offer these benefits:

1. Servicing a request with an existing thread is faster than waiting to create a thread.
2. A thread pool limits the number of threads that exist at any one point. This is particularly important on systems that cannot support a large number of concurrent threads.
3. Separating the task to be performed from the mechanics of creating the task allows us to use different strategies for running the task. For example, the task could be scheduled to execute after a time delay or to execute periodically.



Thank you